

Manual Técnico - Intérprete GoLite

Universidad de San Carlos de Guatemala Facultad de Ingeniería - Ingeniería en Ciencias y Sistemas

Organización de Lenguajes y Compiladores 1

Angel Sebastian Mazariegos Pérez - 202300583

Arquitectura del Sistema

El intérprete de GoLite está desarrollado bajo el paradigma de programación orientada a objetos en Java, usando Visitor como patrón de diseño. Su flujo de ejecución se divide en tres fases principales: análisis léxico, análisis sintáctico y recorrido del Árbol de Sintaxis Abstracta (AST).

1. Análisis Léxico (JFlex)

El escáner fue construido utilizando JFlex. Se encarga de transformar la cadena de caracteres de entrada en una secuencia de tokens válidos.

Implementación del registro de Tokens: Para solventar el problema de sincronización de memoria entre la interfaz gráfica y el analizador, se implementó un helper único en JFlex que inyecta directamente cada token reconocido en una memoria global estática antes de pasarlo al parser:

Java

// Helper en GoLiteLexer.jflex

```
private Symbol token(int symType, String typeName) {  
    // Registro global en memoria estática  
    com.golite.reports.ReportCollector.addToken(  
        new Token(yyline + 1, yycolumn + 1, yytext(), typeName)  
    );  
    return new Symbol(symType, yyline + 1, yycolumn + 1);  
}
```

2. Análisis Sintáctico (CUP)

El parser fue construido utilizando CUP. Implementa una gramática LALR para validar la estructura del código y construir el AST.

Manejo de Structs y Asignaciones Compuestas: La gramática fue expandida para soportar asignaciones directas a atributos de estructuras, mutación por referencia y asignación del valor nulo universal para estructuras dinámicas:

Java

// Snippet de GoLiteParser.cup

```
FieldAssignStmt ::= IDENTIFICADOR:id PUNTO IDENTIFICADOR:campo MAS_IGUAL
Expresion:expr
```

```
{: RESULT = new FieldAssignStmt(id, campo, "+=", expr); :}
```

3. Ejecución y Modo Pánico (Interpreter)

La clase Interpreter.java implementa el patrón de diseño Visitor para recorrer los nodos del AST generados por CUP.

Para garantizar la estabilidad del compilador frente a errores semánticos, se implementó un "Modo Pánico" a nivel de sentencias (Statement). Si una instrucción falla (por ejemplo, en casos de división por cero o tipos incompatibles), el intérprete atrapa la excepción, la registra en el ReportCollector, y continúa el ciclo de ejecución sin detenerse abruptamente:

Java

// Snippet de Interpreter.java

```
for (Statement stmt : block.getStatements()) {
    try {
        stmt.accept(this);
    } catch (GoLiteException e) {
        // Modo Pánico: Reportar error semántico y continuar
        ReportCollector.addError(ErrorType.SEMANTIC, 0, 0, e.getMessage());
    }
}
```

En el repositorio, se encuentra un archivo .bnf en el que se explica la gramática usando dicho formato.